

ARTI 403 — Image Processing

Lab 3 — Image Manipulations using OpenCV

Student name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1
College	Computer Science and Information Technology
Department	Computer Engineering
University	Imam Abdulrahman Bin Faisal University
Date	14 September 2026

1. Objective

Outcome #1: Explain how digital images are represented and manipulated in a computer, and apply geometric transformations to images.

This lab covers reading, displaying and saving images with OpenCV, converting between colour spaces, and applying two families of transformation. Geometric transformations change where each pixel sits; intensity transformations change the value each pixel holds. The distinction organises the whole report.

2. Tools

Component	Version / detail
Python	3
IDE	Jupyter Notebook (via Visual Studio Code)
Libraries	OpenCV, NumPy, Matplotlib, scikit-image
Test image	512 x 512 colour image, saved as images/input.jpg

3. Reading, displaying and saving an image

3.1 Reading

`cv2.imread` returns a plain NumPy array of shape (512, 512, 3) and type `uint8` — 512 rows, 512 columns and three colour channels, each value an integer between 0 and 255. The channels are stored in **BGR** order, not RGB, so an image passed straight to Matplotlib without conversion appears with its red and blue swapped.

A detail worth recording: when the path is wrong, `cv2.imread` does not raise an exception. It returns `None`, and the failure surfaces later as an unrelated error. Checking the result immediately after reading avoids this.

3.2 Displaying inside a notebook

The manual displays images with the pair below:

```
img = cv2.imread('./images/input.jpg')
cv2.imshow('Input image', img)
cv2.waitKey()
```

This opens a separate operating-system window and blocks until a key is pressed. In a notebook the window often opens behind the browser, the cell remains busy, and closing the window with the mouse rather than a keypress can leave the kernel stuck. Every figure in this report is therefore produced with Matplotlib, which renders inline and does not block.

3.3 Grayscale conversion and saving

There are two routes to a grayscale image: passing `cv2.IMREAD_GRAYSCALE` when reading, or calling `cv2.cvtColor` on an image already in memory. They are visually identical but not byte-identical — the largest difference measured was 8 intensity levels. The first route asks the JPEG decoder to produce grayscale directly from the compressed luminance data, while the second decodes to full colour and then converts. Two rounding paths, two slightly different results.



Figure 1 — the colour input, the grayscale version, and the difference between the two grayscale routes (amplified 30x to make it visible).

Saving was tested in both directions. Writing to `.jpg` and reading the file back does **not** return the same array: the largest change was 2 intensity levels, because JPEG is a lossy format. The same test with `.png` returns the array unchanged, PNG being lossless.

Format	Round-trip identical?	Largest pixel change
JPEG (.jpg)	No — lossy compression	2 levels
PNG (.png)	Yes — lossless	0 levels

4. Colour spaces

OpenCV exposes 374 colour-conversion flags in this version. The manual lists the flags with a line written in Python 2 syntax (`print [x for x in dir(cv2) ...]`), which raises a `SyntaxError` in Python 3 where `print` is a function; the corrected form adds parentheses.

4.1 YUV

YUV is the model used for television broadcasting. It separates brightness from colour: **Y** carries luminance, **U** the blue projection and **V** the red projection. The Y channel is indistinguishable from the grayscale conversion, which confirms that it holds the brightness information. U and V appear flat and mid-grey because they store deviations from the average colour and cluster near 128.

The practical consequence is compression. The eye is far more sensitive to brightness than to colour, so U and V can be transmitted at lower resolution with no visible loss — the chroma subsampling used in JPEG and in video.



Figure 2 — the three YUV channels.

4.2 Comparing four colour spaces

The same image in BGR, YUV, HSV and LAB, each split into its channels. BGR mixes brightness into all three channels, while YUV, HSV and LAB each isolate it in one — which is why those three are preferred when brightness must be processed independently of colour.

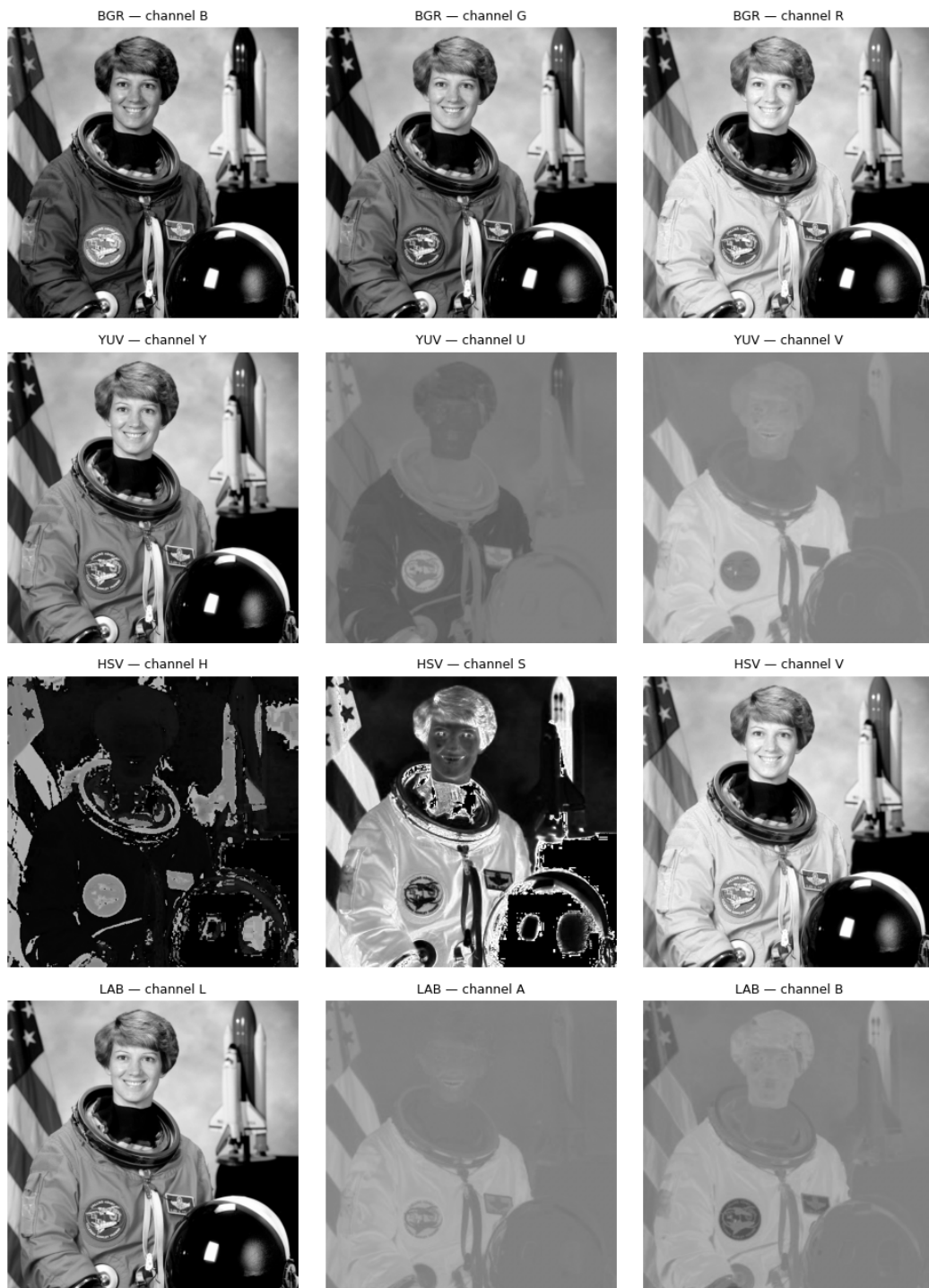


Figure 3 — the image in four colour spaces, channel by channel.

5. Task 1 — Geometric transformations

Geometric transformations change pixel positions and leave values untouched. Scaling, rotation, shear and translation are all affine transformations, expressed as a 2x3 matrix and applied with `cv2.warpAffine`. Because the transformed coordinates are rarely whole numbers, output values must be interpolated from neighbouring input pixels — which makes every one of these operations lossy.

5.1 Increasing the size

`cv2.resize` changes the sampling grid. The interpolation method decides how new pixels are estimated: `INTER_NEAREST` copies the closest pixel and gives a blocky result, `INTER_LINEAR` averages four neighbours, and `INTER_CUBIC` fits a curve over sixteen and produces the sharpest edges. The difference is clearest when a small region is enlarged heavily, as below.

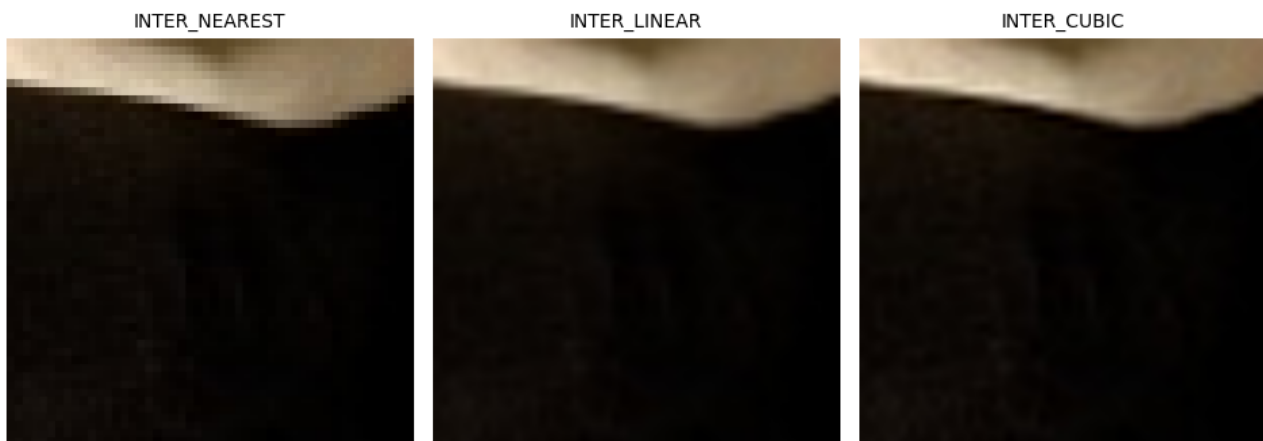


Figure 4 — a 50 x 50 crop enlarged eight times with three interpolation methods.

Enlarging cannot create detail that was never captured; it only estimates intermediate values. All three results are softer than a genuinely higher-resolution photograph would be.

5.2 Rotation by 120 degrees

`cv2.getRotationMatrix2D(center, angle, scale)` builds the rotation matrix. Applying it with the original canvas size **cuts off the corners**, because the rotated rectangle no longer fits inside the original frame. The fix is to compute the bounding box of the rotated rectangle and shift the matrix so that the whole image is preserved — here the canvas grows from 512 x 512 to 699 x 699.

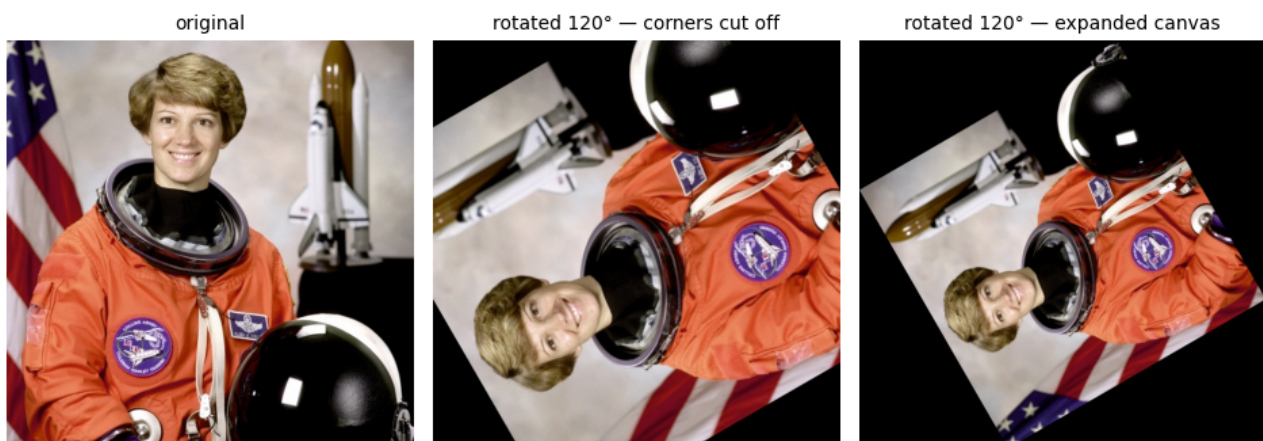


Figure 5 — rotation by 120 degrees with the default canvas (corners lost) and with an expanded canvas.

Rotating by 120 degrees three times returns the original orientation but not the original pixel values. The mean absolute difference over the whole frame is 22.12, but most of that comes from the corners discarded on each pass; measured only inside the inscribed circle, where no data is lost to the canvas, it is 3.0. That residual is interpolation error alone, and it is the clearest evidence that geometric transformations are not reversible.

5.3 Shear

A shear slides each row sideways by an amount proportional to its distance from the origin, turning squares into parallelograms while keeping parallel lines parallel. Horizontal shear uses the matrix $\begin{bmatrix} 1 & k & 0 \\ 0 & 1 & 0 \end{bmatrix}$ and vertical shear $\begin{bmatrix} 1 & 0 & 0 \\ k & 1 & 0 \end{bmatrix}$. The output canvas has to be widened by k times the image height, otherwise the sheared image is clipped.

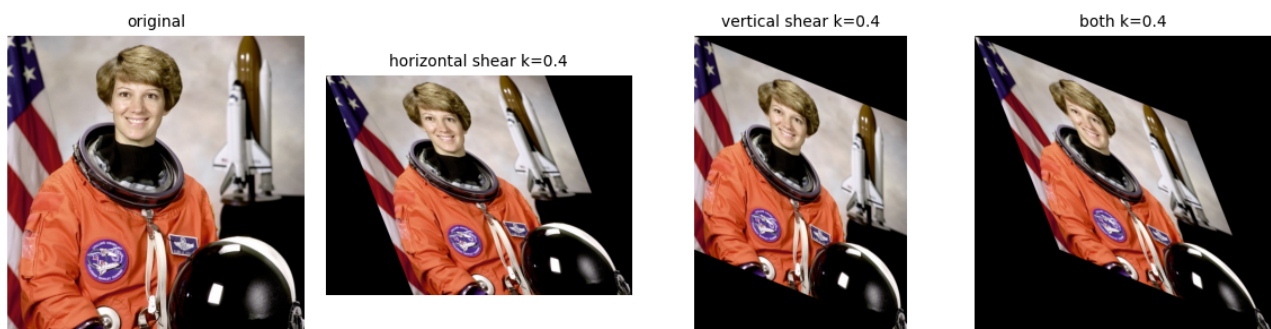


Figure 6 — horizontal, vertical and combined shear with $k = 0.4$.



Figure 7 — all geometric transformations applied in this task.

6. Task 2 — Intensity transformations

Intensity transformations are the mirror image of the previous section: every pixel stays where it is and only its value changes. Each is a function $s = T(r)$ applied independently to every pixel, where r is the input intensity and s the output. All three were applied to the grayscale image.

Transformation	Formula	Effect
Negative	$s = 255 - r$	inverts the range; reveals detail in dark regions
Log	$s = c \log(1 + r)$	expands dark tones, compresses bright ones
Power law	$s = c r^{\gamma}$	$\gamma < 1$ brightens, $\gamma > 1$ darkens

6.1 Negative

The negative is its own inverse: applying it twice returns the original array exactly, which was verified. It is standard practice when reading X-ray film, because detail hidden in dark regions becomes easier for the eye to pick out once those regions are bright.

6.2 Log transformation

The constant c is chosen so that the largest input maps exactly to 255, giving $c = 255 / \log(1 + 255) = 45.99$. The log curve is steep for small values and flat for large ones, so it stretches the dark range and compresses the bright range, brightening the image overall and revealing shadow detail. The inverse log does the opposite.

One implementation detail cost some debugging: writing $1 + r.\max()$ where $r.\max()$ is a `uint8` of value 255 wraps round to 0, making the logarithm undefined. The maximum has to be cast to a wider type first.

6.3 Power law and the choice of gamma

The task asks for a gamma that increases the contrast. Rather than guessing, standard deviation was used as a numerical measure of contrast and computed across gamma values from 0.2 to 3.0. The maximum occurs at gamma = 0.6, raising the standard deviation from 75.09 to 76.7.

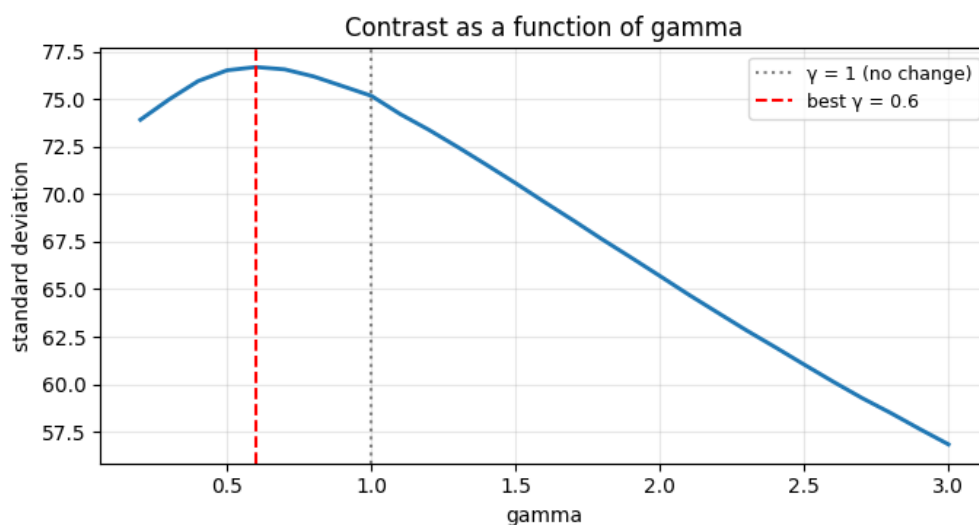


Figure 8 — contrast, measured as standard deviation, against gamma.

The gain is small, and that is a fact about this image rather than a failure of the method. The original already spans the full range from 0 to 255 with a broad histogram, so a single monotonic curve has little headroom. Gamma does not create contrast; it **redistributes** it, taking from the bright tones and giving to the dark ones or the reverse. The overall standard deviation therefore moves only slightly while the local effect on shadows or highlights is large.



Figure 9 — all intensity transformations applied in this task.

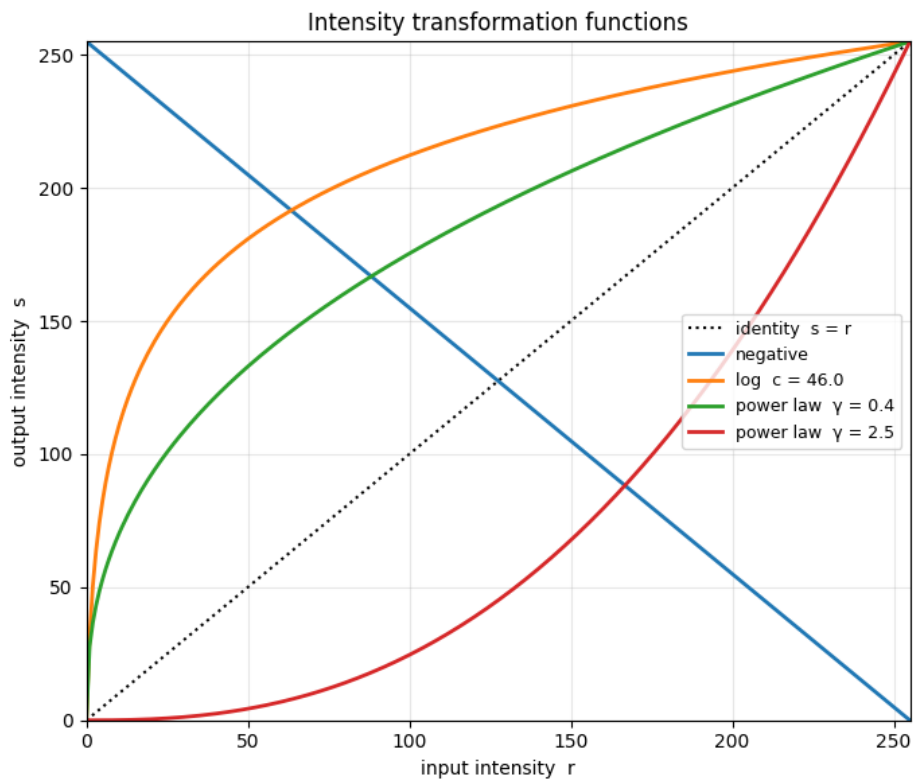


Figure 10 — the transformation curves. The dotted diagonal is the identity; curves above it brighten, curves below it darken, and steeper sections receive more contrast.

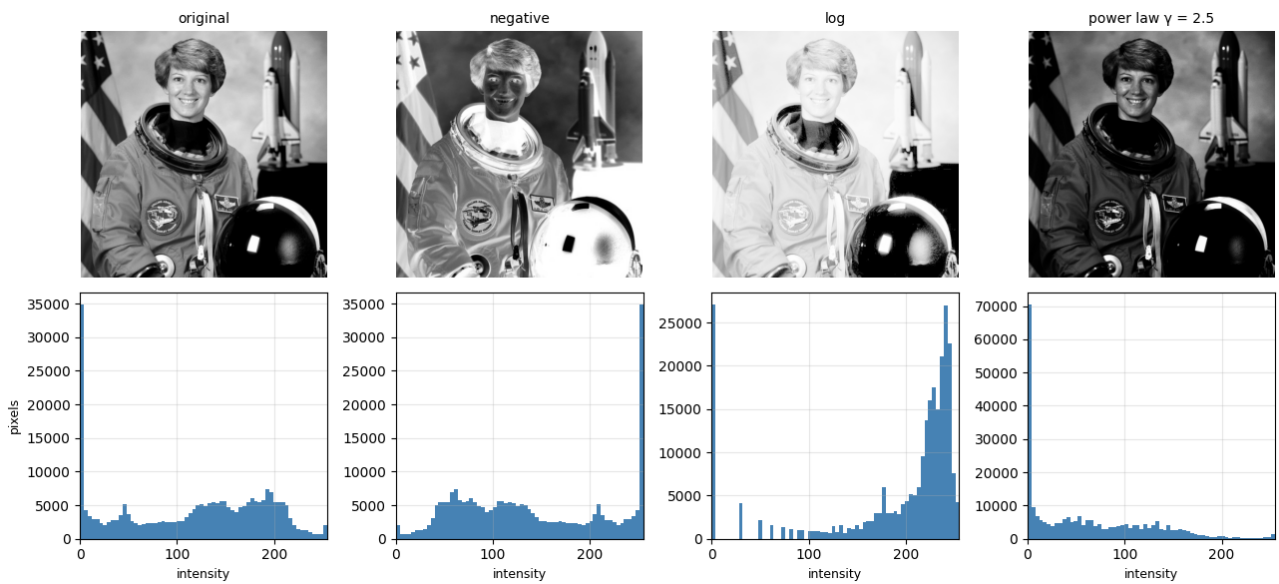


Figure 11 — each transformation with its histogram, showing how the distribution of intensities is reshaped.

7. Observations and conclusion

7.1 What the two families have in common

Both are ordinary array operations. A geometric transformation is a remapping of indices; an intensity transformation is a function applied element-wise to values. Neither requires anything conceptually beyond what a NumPy array already supports, which is the central point of the outcome: an image is an array, and image processing is array processing.

7.2 Points that required care

Issue	What happens	How it was handled
BGR vs RGB	colours appear swapped in Matplotlib	convert with <code>cv2.cvtColor</code> before displaying
<code>cv2.imread</code> on a bad path	returns <code>None</code> instead of raising	check the result immediately after reading
<code>cv2.imshow</code> in Jupyter	blocking window, can freeze the kernel	display with Matplotlib instead
Rotation canvas	corners fall outside the frame	compute the bounding box and shift the matrix
uint8 in the log formula	$1 + 255$ wraps round to 0	cast the maximum to a wider type
JPEG round-trip	saved values differ on reading back	use PNG when the exact values matter

7.3 Conclusion

Reading an image with OpenCV produces a NumPy array in BGR order, and saving it to a lossy format changes the values it holds. Colour spaces such as YUV separate brightness from colour, which is what makes modern image and video compression possible. Geometric transformations are affine matrices applied with interpolation, so they lose information each time they are applied and crop the image unless the output canvas is computed deliberately. Intensity transformations leave positions untouched and reshape the distribution of values, and choosing their parameters is better done by measuring an objective quantity than by eye.

8. References

Image transformations using OpenCV in Python — GeeksforGeeks
<https://www.geeksforgeeks.org/image-transformations-using-opencv-in-python/>

Image transformations using OpenCV in Python — The Python Code
<https://www.thepythoncode.com/article/image-transformations-using-opencv-in-python>

Basic intensity transformation functions with OpenCV — Medium
<https://medium.com/@khanhson0811/basic-intensity-transformation-functions-and-example-with-opencv-1a56932f34ac>

OpenCV documentation — geometric image transformations
<https://docs.opencv.org/>